

ARCOS Group

uc3m | Universidad **Carlos III** de Madrid

L3: Fundamentals of assembler programming (5) Computer Structure

Bachelor in Computer Science and Engineering

Bachelor in Applied Mathematics and Computing

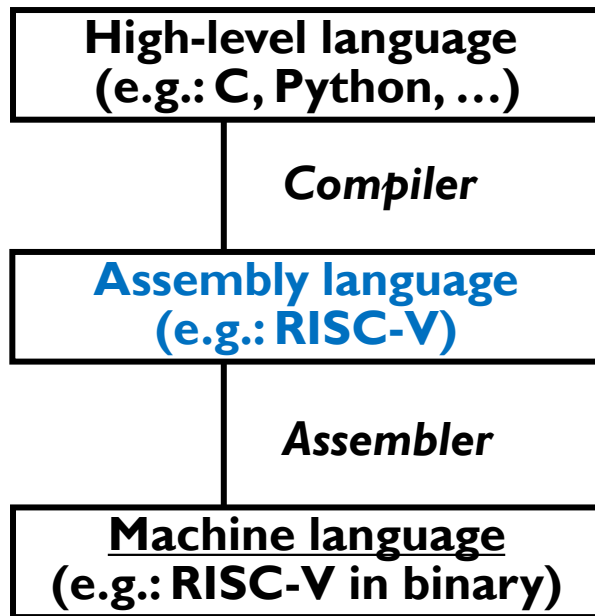
Dual Bachelor in Computer Science and Engineering and Business Administration



Contents

1. Basic concepts on assembly programming
2. RISC-V 32 assembly, memory model and data representation
3. Floating point in assembly
4. Procedure calls and stack convention
5. **Instruction formats**, addressing modes, and instruction sets

Different language levels



```
temp = v[k];  
v[k] = v[k+1];  
v[k+1] = temp;
```

```
lw    t0, 0(x2)  
lw    t1, 4(x2)  
sw    t1, 0(x2)  
sw    t0, 4(x2)
```

```
0000 1001 1100 0110 1010 1111 0101 1000  
1010 1111 0101 1000 0000 1001 1100 0110  
1100 0110 1010 1111 0101 1000 0000 1001  
0101 1000 0000 1001 1100 0110 1010 1111
```

- ▶ An **assembly instruction** corresponds to a **machine instruction**
 - ▶ Example:
 - ▶ Assembly: `add x5, x6, x2`
 - ▶ Machine: `0x002302B3`

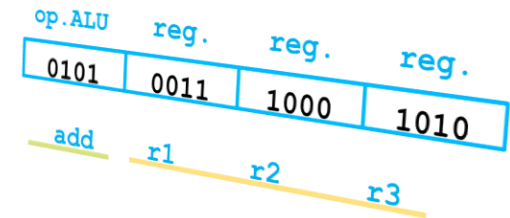
Instruction and pseudoinstruction

RISC-V₃₂

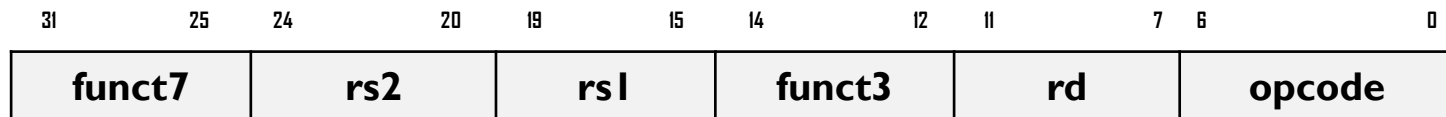
Reminder

- ▶ A **pseudo-assembler** instruction corresponds to **one or several machine instructions**.
 - ▶ Example 1:
 - ▶ The instruction: `mv x2, x1`
 - ▶ It is equivalent to: `add x2, zero, x1`
 - ▶ Example 2:
 - ▶ The instruction: `li t1, 0x00800010`
 - ▶ It does not fit in 32 bits, but it can be used as a pseudoinstruction.
 - ▶ It is equivalent to:
 - `lui t1, 0x00800`
 - `ori t1, t1, 0x010`
- ▶ An **assembly instruction** corresponds to a **machine instruction**
 - ▶ Example:
 - ▶ Assembly: `add x5, x6, x2`
 - ▶ Machine: `0x002302B3`

Instruction format

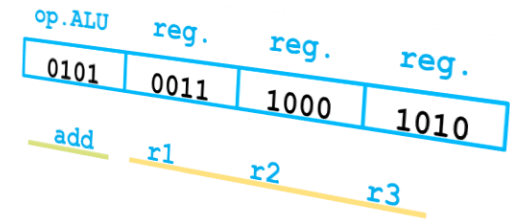


- ▶ A **machine instruction** is **coded in binary**:
 - ▶ Size **fits one** or **more words**
 - ▶ An instruction is divided into **fields**
 - ▶ Each field encodes an element that includes the statement
 - ▶ There may be implicit elements
 - ▶ Example of fields in a RISC-V instruction:



- ▶ The **format** specifies, for each field of the instruction:
 - ▶ The **meaning** of **each field**
 - ▶ **Encoding used** in each field
 - ▶ Binary, one's complement, two's complement, etc.
 - ▶ The **number of bits** in each field
 - ▶ The size of the fields limits the number of values to be encoded.

Instruction format



- ▶ A **machine instruction** is self-contained and **includes**:
 - ▶ **Operation code**
 - ▶ **Operands** (value or location of value to use)
 - ▶ **Result** (location to save)
 - ▶ **Address** of the **next instruction**
 - ▶ Implicit: $PC \leftarrow PC + '4'$ (point to the following 32-bit instruction)
 - ▶ Explicit: j 0x01004 (modifies the PC)
- ▶ Usually:
 - ▶ One **architecture offers a few instruction formats**.
 - ▶ Simplicity in the design of the control unit.
 - ▶ **Fields** of the **same type** always of **equal length**.
 - ▶ **Selection together** with the **operation code** (e.g. add, addi).
 - ▶ **Usually the first field.**

Format length

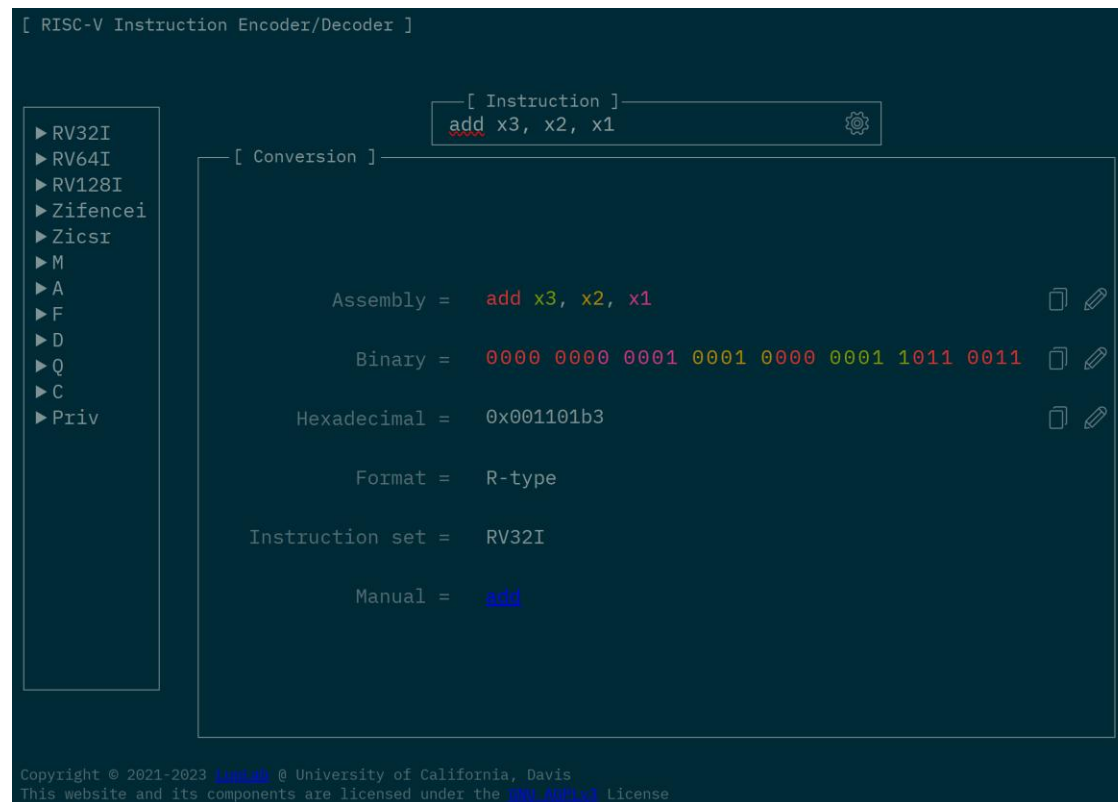
- ▶ The **format length** is number of bits to encode the instruction
 - ▶ The size of an instruction is usually one word (or multiples words)
 - ▶ In RISC-V₃₂ the size of all instructions is one word (32 bits)
- ▶ Two types:
 - ▶ **Fixed/Unique length:**
 - ▶ All instructions with the same size
 - ▶ Examples:
 - MIPS32 (32 bits), PowerPC (32 bits), ...
 - ▶ **Variable length:**
 - ▶ Different instructions can have different sizes
 - ▶ How to know the instruction length? → Op. code
 - ▶ Examples:
 - IA32 (Intel processors): variable number of bytes

Example: instruction format for RISC-V

	31				25		24				20		19				15		14				12		11				7		6				0							
R	funct7						rs2						rs1						funct3						rd						opcode											
I	imm[11:0]																		rs1						funct3						rd						opcode					
UI	imm[31:12]																								rd						opcode											
S	imm[11:5]						rs2						rs1						funct3						imm[4:0]						opcode											
B	[12]	imm[10:5]						rs2						rs1						funct3						imm[4:1]						[11]	opcode									
J	[20]	imm[10:1]												[11]	imm[19:12]												rd						opcode									

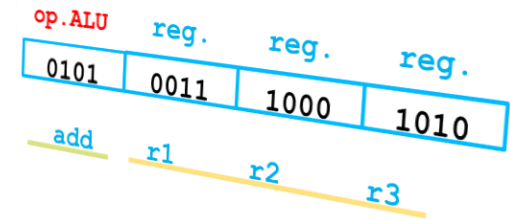
- **opcode** (7 bits): partially indicates the type of instruction format.
 - **R**egister, **I**mmEDIATE, **U**pper Immediate, **S**TORE, **B**RANCH, **J**UMP
- **funct7+funct3** (10 bits): together with opcode, describe the op. to perform.
- **rs1** (5 bits): specifies the register as first operand.
- **rs2** (5 bits): specifies the register as second operand.
- **rd** (5 bits): specifies the target register.

Instruction format for RISC-V

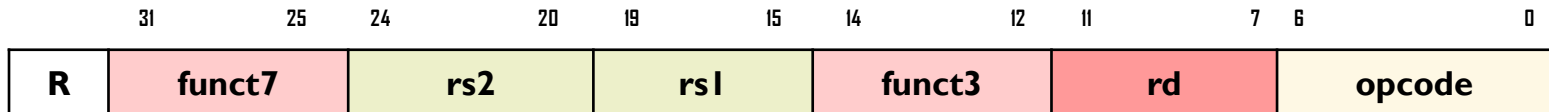


- ▶ <https://luplab.gitlab.io/rvcodecs/>
- ▶ In 32-bit and 64-bit RISC-V, instructions are 32 bits long; the only difference is the size of the registers.

Operation code



- ▶ Fixed size:
 - ▶ n bits $\rightarrow 2^n$ operation codes
 - ▶ m operation codes $\rightarrow \lceil \log_2 m \rceil$ bits.
- ▶ Extension fields
 - ▶ RISC-V (arithmetic-logic instructions)
 - ▶ $Op = 0$; the instruction is encoded in `functX`



- ▶ Variable sizes:
 - ▶ More frequent instructions = shorter sizes

Locations of operands

op. ALU	reg.	reg.	reg.
0101	0011	1000	1010
add	r1	r2	r3

1. In the instruction

li t0, 0x123

2. In registers (processor)

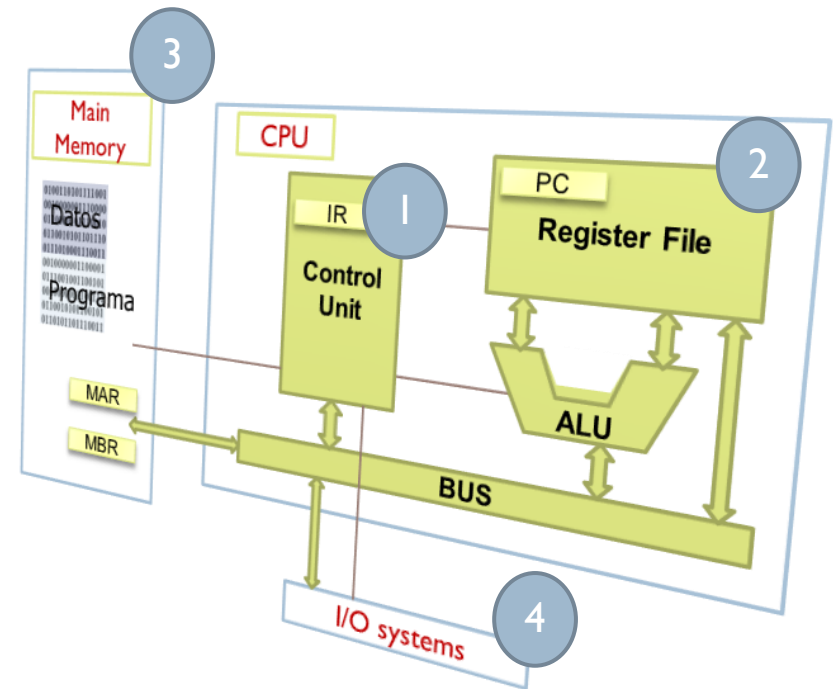
li t0, 0x123

3. Main memory

lw t0, address(x0)

4. Input/output modules

in t0, 0xFEB



Locations of operands

1. In the instruction

```
li t0, 0x123
```

2. In registers (processor)

```
li t0, 0x123
```

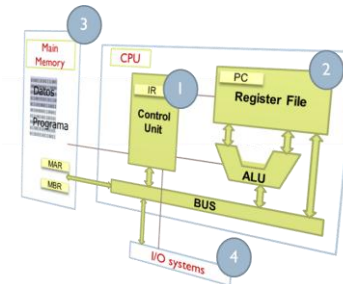
3. Main memory

```
lw t0, address(x0)
```

- **num(registro)**: represents the address obtained by summing num with the address stored in the register

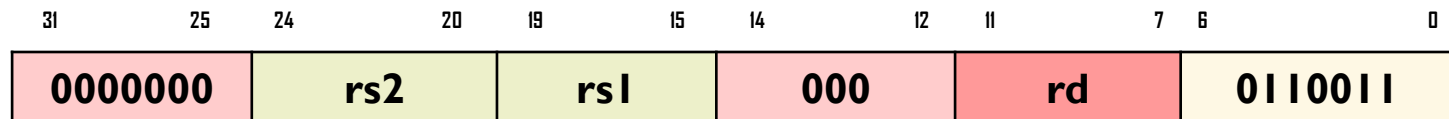
4. Input/output modules

```
in t0, 0xFEB
```



Example instruction and associated format in RISC-V

► `add rd, rs1, rs2`



[Instruction]
`add t0, t1, t2`



Assembly = `add x5, x6, x7`

Binary = `0000 0000 0111 0011 0000 0010 1011 0011`

Hexadecimal = `0x007302b3`

Exercise

- ▶ A 16-bit computer has an instruction set of 60 instructions and a register file of 8 registers.

The following is requested:

Define the format of this instruction: **ADDx RI R2 R3**, where RI, R2 and R3 are registers.

Exercise (solution)

word-> 16 bits
60 instructions
8 registers (in RB)
ADDx R1(reg.), R2(reg.), R3(reg.)

- ▶ 16-bit word defines the size of the instruction

16 bits



Exercise (solution)

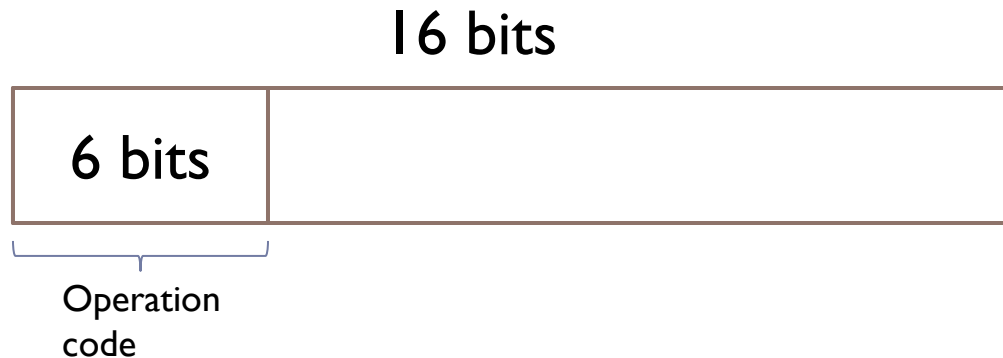
word-> 16 bits

60 instructions

8 registers (in RB)

ADDx R1(reg.), R2(reg.), R3(reg.)

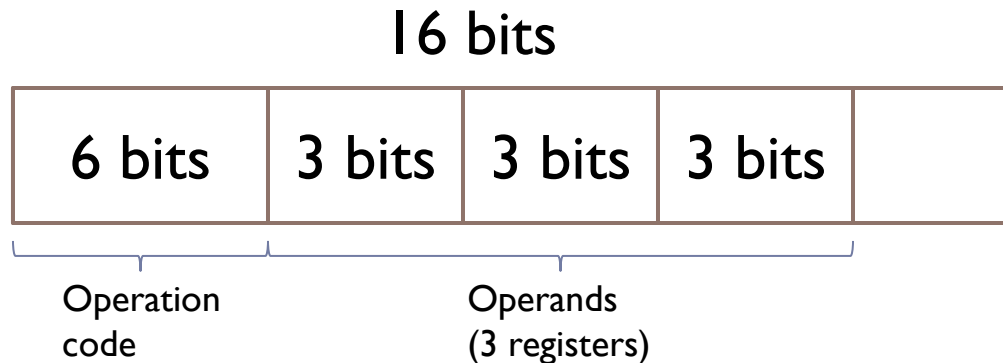
- ▶ To encode 60 instructions, 6 bits are required for the operation code (minimum)



Exercise (solution)

word-> 16 bits
60 instructions
8 registers (in RB)
ADDx R1(reg.), R2(reg.), R3(reg.)

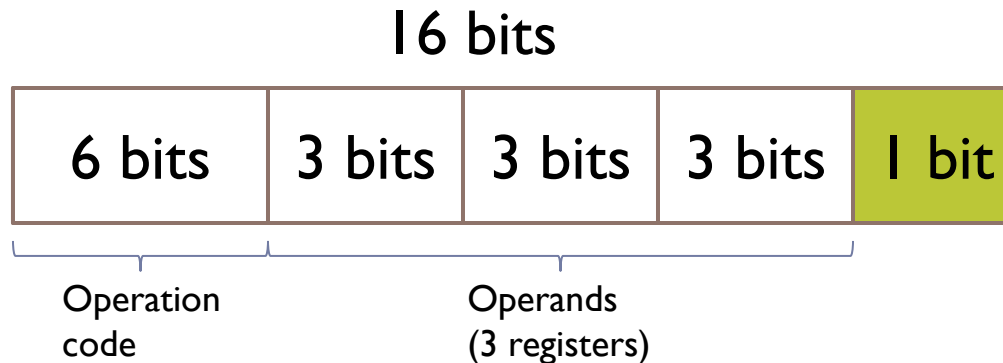
- For 8 registers, 3 bits are required (minimum)



Exercise (solution)

word-> 16 bits
60 instructions
8 registers (in RB)
ADDx R1(reg.), R2(reg.), R3(reg.)

- ▶ 1 bit left over ($16 - 6 - 3 - 3 - 3 = 1$), used for padding



Contents

1. Basic concepts on assembly programming
2. RISC-V 32 assembly, memory model and data representation
3. Floating point in assembly
4. Procedure calls and stack convention
5. Instruction formats, **addressing modes**, and instruction sets

Addressing modes

- ▶ The **addressing mode** is a procedure for determining the **location** of an operand, a result or an instruction

- ▶ Implicit
- ▶ Immediate
- ▶ Direct 
 - to register
 - to memory
- ▶ Indirect 
 - to register
 - to memory
- ▶ Relative 
 - to index register
 - base register
 - to PC
 - to stack

Modos de direccionamiento en RISC-V

- ▶ Immediate value
- ▶ Direct
 - ▶ To memory address
 - ▶ To register xr
- ▶ Indirect
 - ▶ To memory
 - ▶ To register (xr)
- ▶ Relative to
 - register offset(xr)
 - stack offset(sp)
 - PC beq ... label l

Addressing modes

- ▶ The **addressing mode** is a procedure for determining the **location** of an operand, a result or an instruction

- ▶ **Implicit**
- ▶ **Immediate**
- ▶ **Direct** 
 - to register
 - to memory
- ▶ **Indirect** 
 - to register
 - to memory
- ▶ **Relative** 
 - to index register
 - base register
 - to PC
 - to stack

Implicit addressing

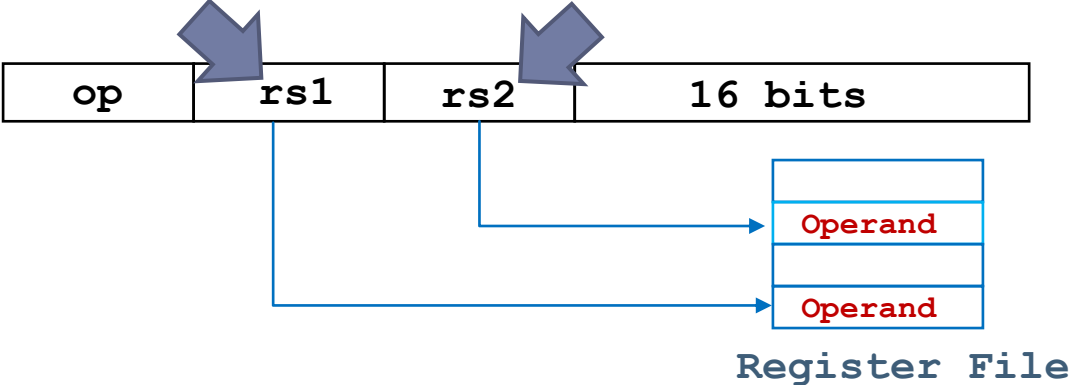
Description	▶ The operand is not coded in the instruction but is part of the instruction.
Example	▶ <code>auipc a0, 0x12345</code> ▶ $a0 = PC + (0x12345 \ll 12)$. ▶ <code>a0</code> is one operand, PC is the other (implicit) 
(V/I) Advantages / Disadvantages	✓ Fast: no need to access memory. ✗ But it is only possible in a few cases.

Immediate addressing

Description	▶ The operand is part of the instruction.				
Example	▶ li a0, 0x4f5 l ▶ Loads in register a0 the immediate value 0x4f5 l. ▶ The value 0x00004f5 l is in an immediate field. <table><tr><td>op</td><td>rs</td><td></td><td>16 bits</td></tr></table>	op	rs		16 bits
op	rs		16 bits		
(V/I) Advantages / Disadvantages	✓ It is fast: no need to access memory. ✗ Value does not always fit in a word: ▶ It does not fit in 32-bits, it is equivalent to: □ lui t1, 0x87654 □ ori t1, t1, 0x321				

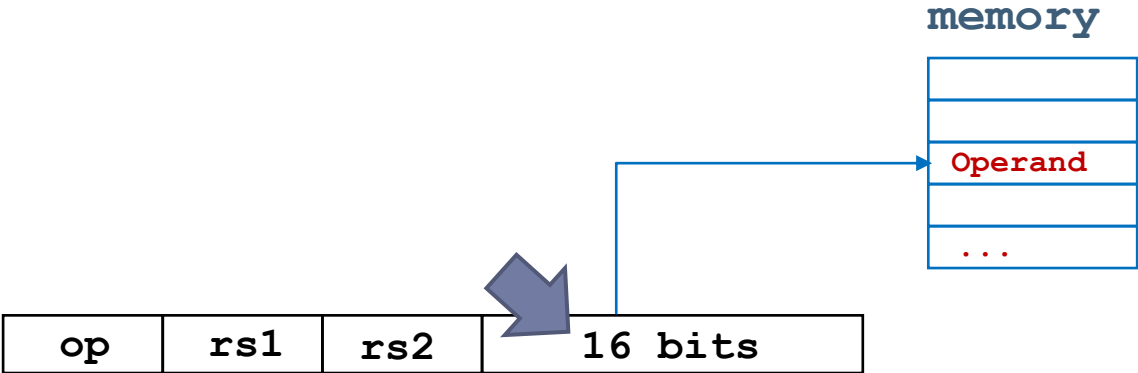
Direct to register addressing

Register addressing

Description	▶ Operand is in a register.
Example	▶ <code>mv a0, a1</code> ▶ Copies into register <code>a0</code> the value in register <code>a1</code>. ▶ The identifier of <code>a0</code> and <code>a1</code> is encoded in the instruction. 
(V/I) Advantages / Disadvantages	✗ The number of registers is limited ✓ Access to registers is fast ✓ The number of registers is small => few bits for encoding, shorter instructions

Direct to memory addressing

Direct addressing

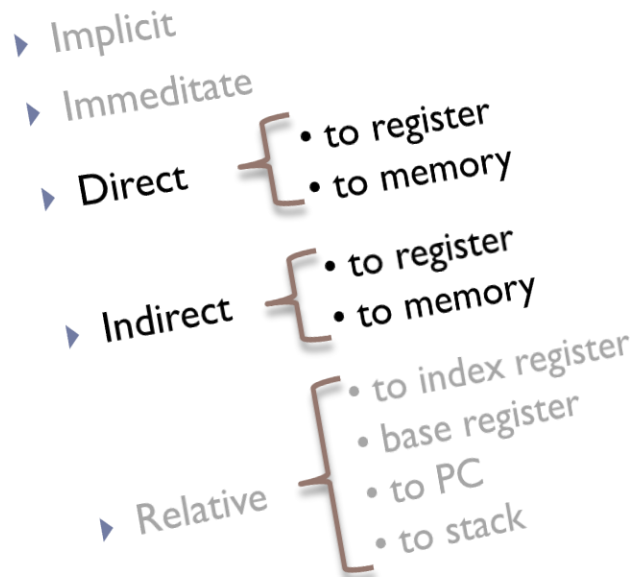
Description	▶ The operand is in memory, and the address is coded in the instruction (not available on RISC-V).
Example	▶ LD .RI #0xFFF0 # IEEE 694 ▶ Loads in RI the word stored in 0xFFF0. 
(V/I) Advantages / Disadvantages	✗ Memory access is slower compared to registers ✗ Long addresses => longer instructions ✓ Access to a large address space (capacity larger than R.F.)

Addressing modes

- ▶ The **addressing mode** is a procedure for determining the **location** of an operand, a result or an instruction

- ▶ Implicit
- ▶ Immediate
- ▶ Direct
 - to register
 - to memory
- ▶ **Indirect**
 - **to register**
 - **to memory**
- ▶ Relative
 - to index register
 - base register
 - to PC
 - to stack

Direct vs. indirect addressing



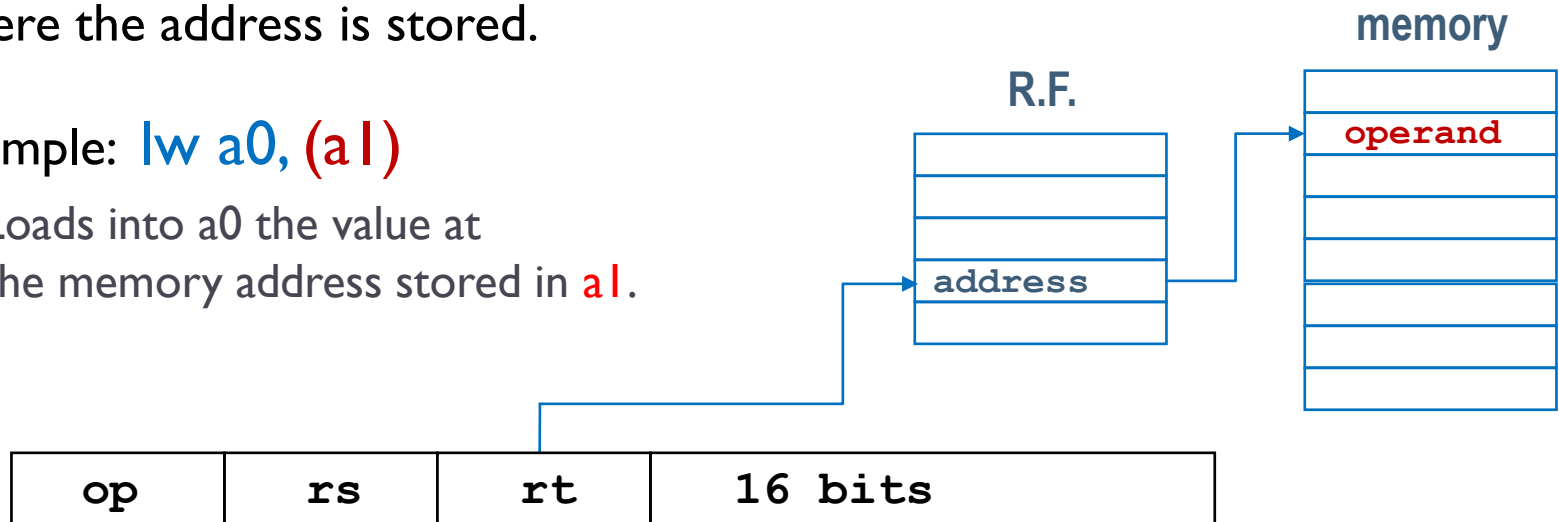
- ▶ Direct addressing indicates **where the operand is located**:
 - ▶ In which register or in which memory position
- ▶ Indirect indicates where **the address of the operand is located**:
 - ▶ This address must be accessed in memory
 - ▶ A level (or several) of addressing is incorporated.

Register indirect addressing

- ▶ The instruction has the register where the address is stored.

- ▶ Example: `lw a0, (a1)`

- ▶ Loads into a0 the value at the memory address stored in a1.



- ▶ V/I

- ✓ Wide address space, short instructions

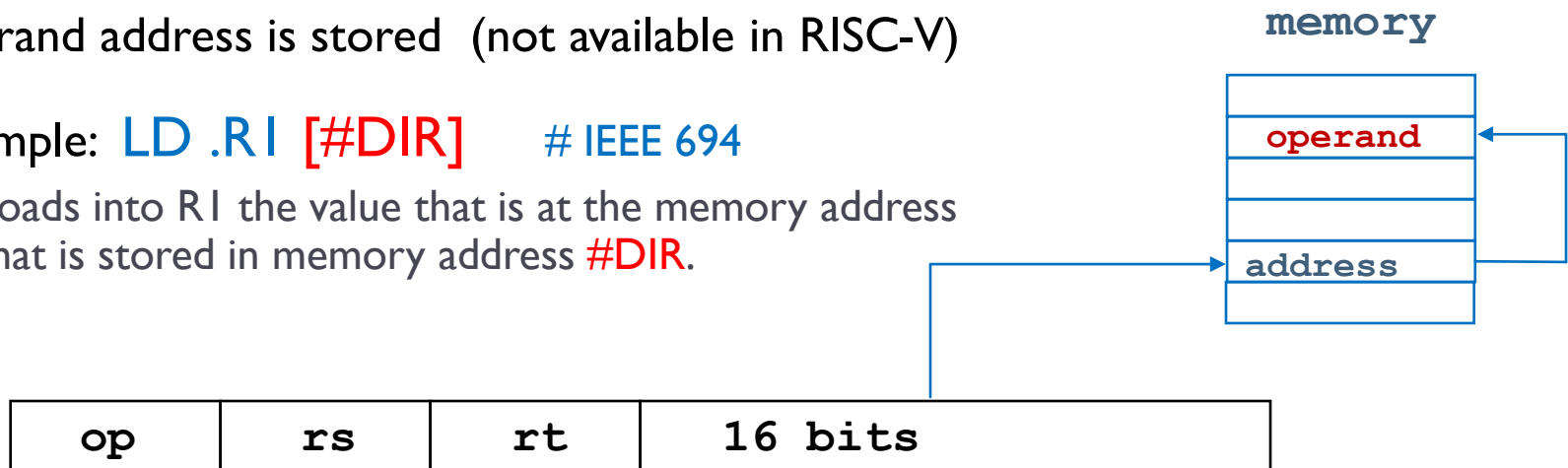
- ▶ Pseudo-instruction equivalent to `lw a0, 0(a1)`

Indirect addressing

- ▶ The instruction has the address where the operand address is stored (not available in RISC-V)

▶ Example: `LD .RI [#DIR] # IEEE 694`

- ▶ Loads into RI the value that is at the memory address that is stored in memory address `#DIR`.



- ▶ V/I

- ✓ Large address space
- ✓ Addressing can be nested, multilevel or cascading
 - ▶ Example: `LD .RI [[[.RI]]]`
- ✗ May require several memory accesses
- ✗ slower instructions to execute

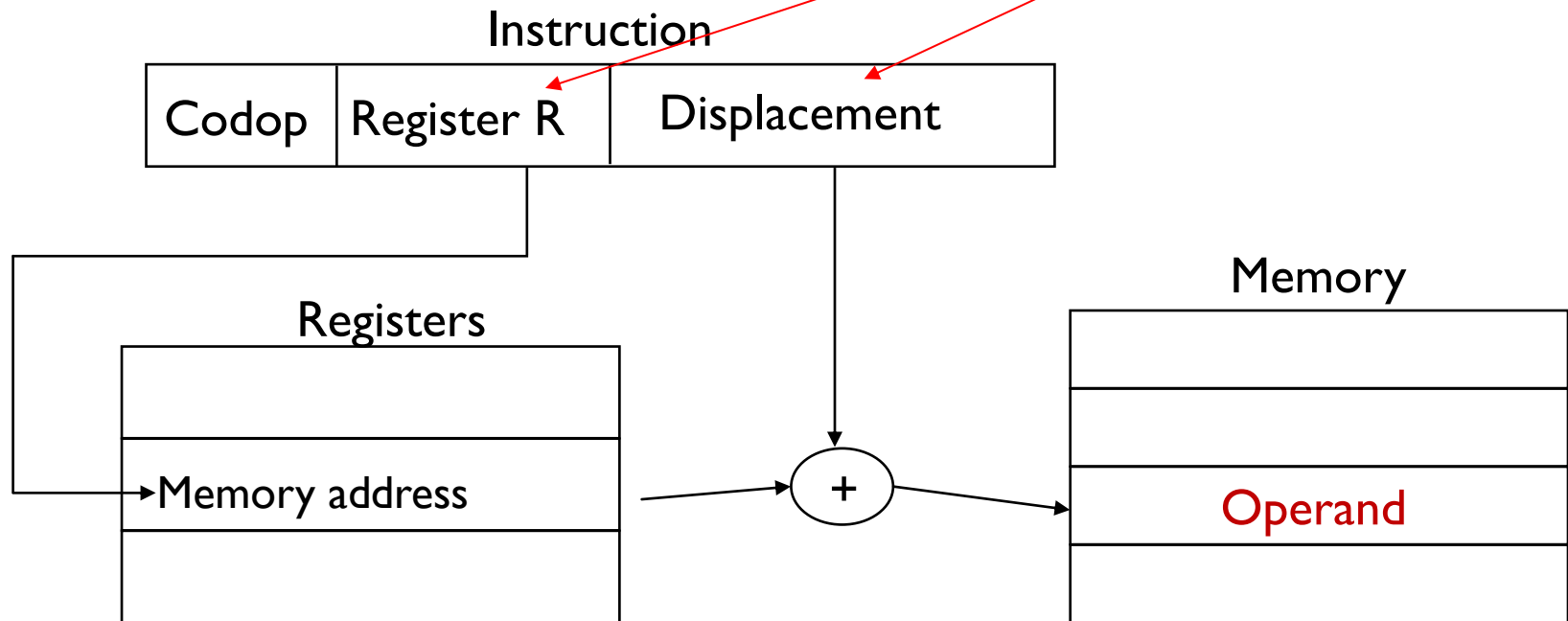
Addressing modes

- ▶ The **addressing mode** is a procedure for determining the **location** of an operand, a result or an instruction

- ▶ Implicit
- ▶ Immediate
- ▶ Direct 
 - to register
 - to memory
- ▶ Indirect 
 - to register
 - to memory
- ▶ **Relative** 
 - **to index register**
 - **base register**
 - **to PC**
 - **to stack**

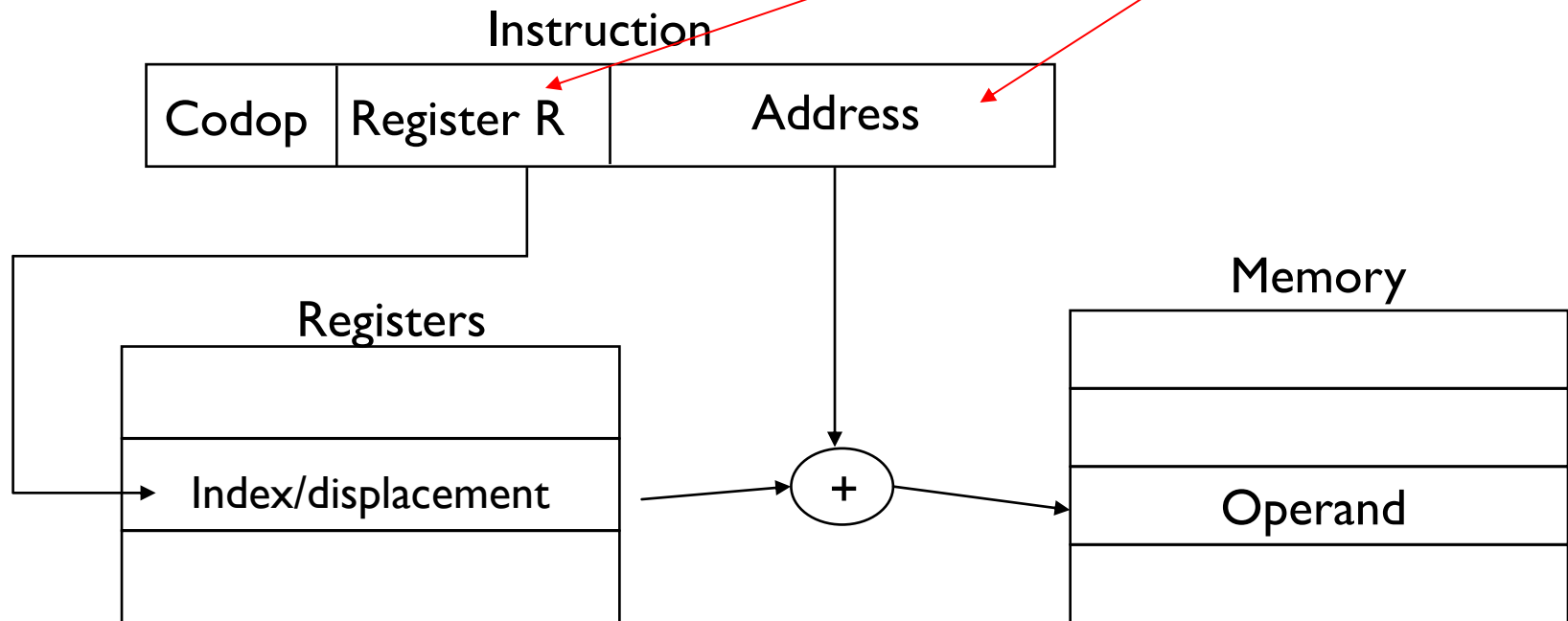
Base-register addressing

- Example: `lw a0, 12(tl)`
 - Loads into a0 the contents of the memory position given by $tl + 12$
 - Uses two fields of the instruction, **tl has the base address**



Index-register addressing

- Example: `lw a0, dir(tl)`
 - Loads into a0 the contents of the memory position given by $tl + dir$
 - It uses two fields: **tl** represents the displacement (**index**) with respect to the address **dir**



Utility: access to vectors

```
int v[5] ;
```

```
main ( )
```

```
{
```

```
    v[3] = 5 ;
```

```
    v[4] = 8 ;
```

```
    v[1] = 3 ;
```

```
}
```

```
.data
```

```
    v: .zero 20    # 5int*4bytes/int
```

```
.text
```

```
main:
```

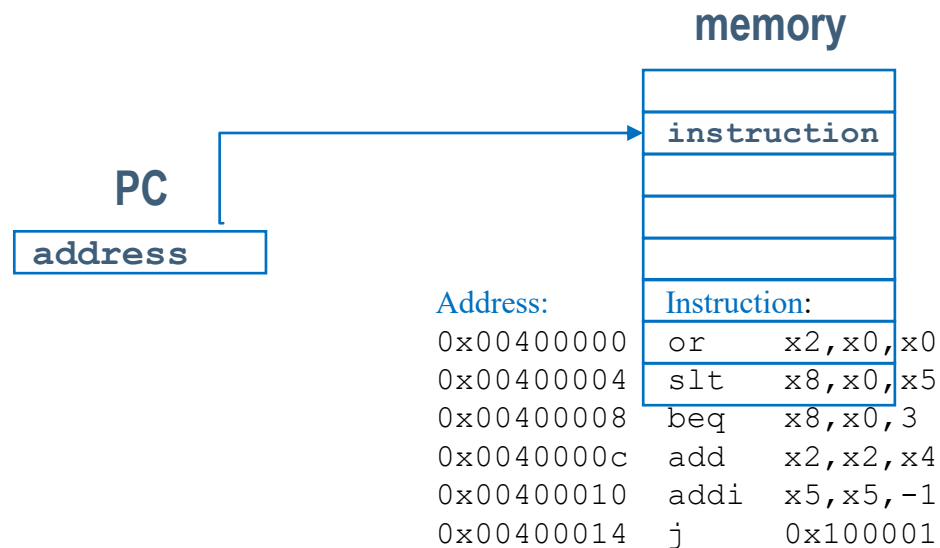
```
la    t0, v
li    t1, 5
sw    t1, 12(t0)
```

```
li    t0, 16
li    t1, 8
sw    t1, v(t0) # (!)
```

```
la    t0, v
addi  t0, t0, 4
li    t1, 3
sw    t1, 0(t0)
```

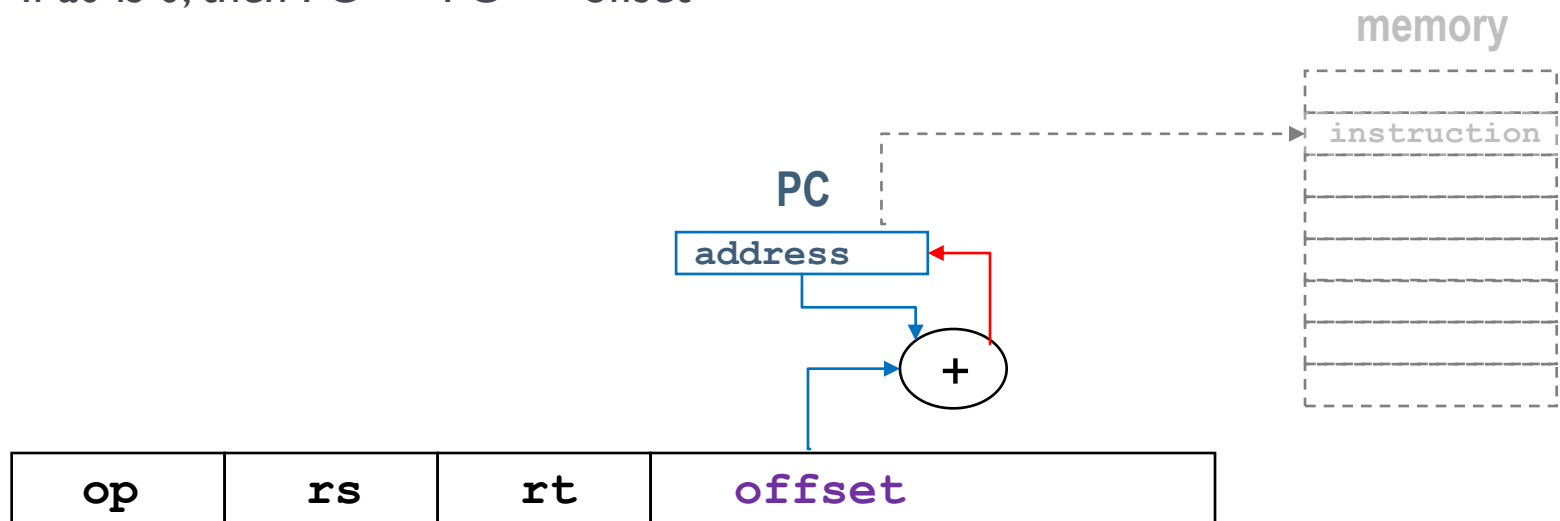
PC-relative addressing

- ▶ Registers of 32 bits (4 bytes) in a 32-bits computer
- ▶ PC stores the address of the next instruction to be executed
 - ▶ Points to a word (4 bytes) in memory with the instruction to be executed
 - ▶ PC in a 32-bits computer is updated by default as $PC = PC + 4$



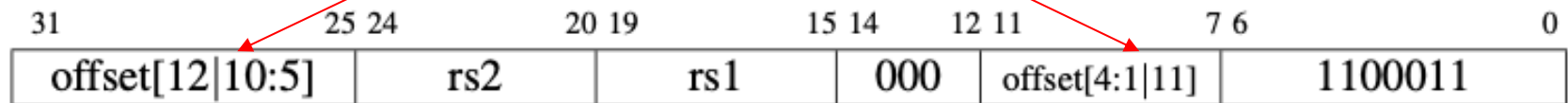
PC-relative addressing

- ▶ Example: `beq a0, x0, label`
 - ▶ The label is encoded as the displacement from the memory address where this instruction is, to the memory position indicated in `label`.
 - ▶ **Label encoded as offset** (address $\rightarrow$ number of instructions to jump)
 - ▶ If `a0` is 0, then $PC \leq PC + \text{“offset”}$



PC-relative addressing in RISC-V

- ▶ The `beq t0, x1, offset` instruction is encoded in the machine instruction:



- ▶ Label must be encoded in the "offset" field as an offset relative to the `beq` instruction at the time of execution (PC points to the first byte of the following instruction)
 - ▶ The offset value can be positive or negative
- ▶ How is PC updated if `t0 == x1`?
 - ▶ If the condition is met:
 - ▶ $PC = PC + \text{"offset"} \rightarrow PC = PC - 4 + \text{se}(\text{offset}[1:12])$
 - ▶ When an instruction is executed, the PC points to the next one

PC-relative addressing in RISC-V

- ▶ What is the value of **fin** when machine code is generated?

CREATOR

```
bucle:    beq    t0, x1, fin
          add    t8, t4, t4
          addi   t0, x0, -1
          j      bucle
fin:      mv     t1 x0
          . . .
```

- ▶ The value of fin is:

- ▶ `fin == 16`

- ▶ Remainder:

- ▶ Label must be encoded as an offset between the address of the jump instruction and the address represented by the label
- ▶ When an instruction is executed, the PC points to the next one.
- ▶ 3 instructions ("addi", "j" and "mv") must be skipped.
- ▶ Each instruction to be skipped is 4 bytes

Used in loops

```
        li    t0, 8
        li    t1, 4
        li    t2, 1
        li    t4, 0
while:   bge   t4, t1, fin
        mul   t2, t2, t0
        addi  t4, t4, 1
        j     while
fin:     mv    t2, t4
```

- ▶ **end** represents the address where the instruction `mv` is stored
- ▶ **while** represents the address where the instruction `bge` is stored

Used in loops

```
        li    t0, 8
        li    t1, 4
        li    t2, 1
        li    t4, 0
while:   bge   t4, t1, fin
        mul   t2, t2, t0
        addi  t4, t4, 1
        j     while
fin:     mv    t2, t4
```

Address	Content
0x0000100	li t0, 8
0x0000104	li t1, 4
0x0000108	li t2, 1
0x000010C	li t4, 0
0x0000110	bge t4, t1, fin
0x0000114	mul t2, t2, t0
0x0000118	addi t4, t4, 1
0x000011C	j while
0x0000120	mv t2, t4

Used in loops

```
        li    t0, 8
        li    t1, 4
        li    t2, 1
        li    t4, 0
while:  bge    t4, t1, fin
        mul   t2, t2, t0
        addi  t4, t4, 1
        j     while
fin:    mv     t2, t4
```

- **fin** encoded as displacement relative to current PC => **12**
 $PC = PC + 3 * 4$
- **while** encoded as displacement relative to current PC => **-16**
 $PC = PC + (-4) * 4$

Address	Content
0x0000100	li t0, 8
0x0000104	li t1, 4
0x0000108	li t2, 1
0x000010C	li t4, 0
0x0000110	bge t4, t1, fin
0x0000114	mul t2, t2, t0
0x0000118	addi t4, t4, 1
0x000011C	j while
0x0000120	mv t2, t4

Used in loops

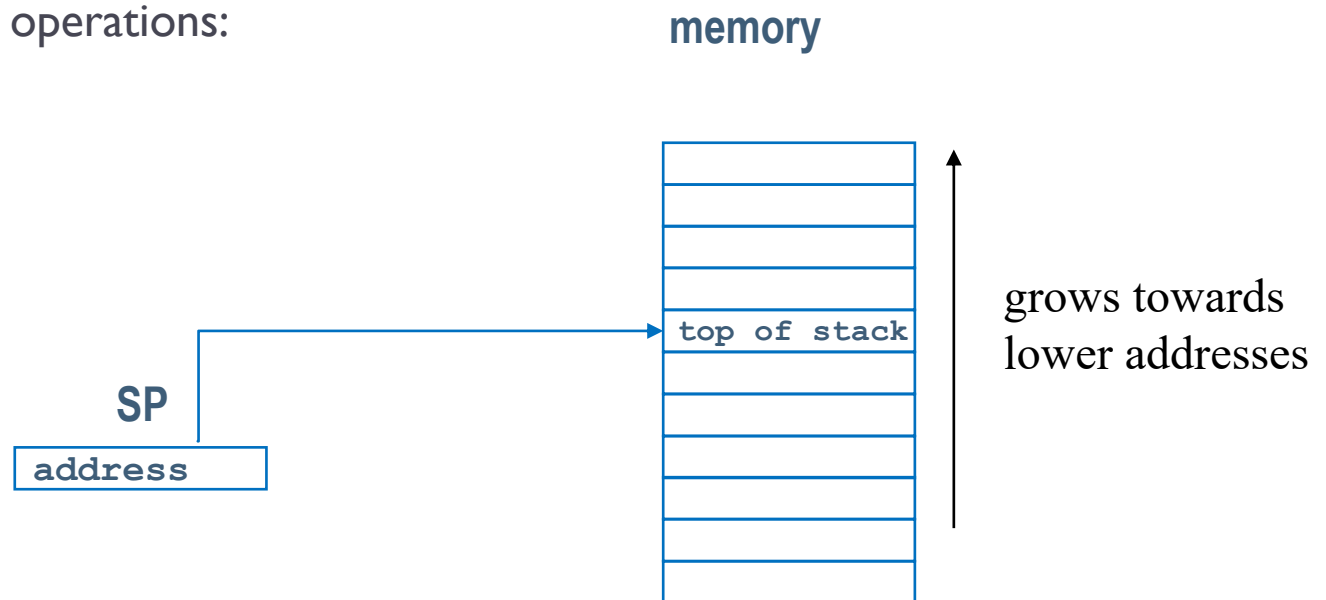
```
        li    t0, 8
        li    t1, 4
        li    t2, 1
        li    t4, 0
while:   bge   t4, t1, fin
        mul   t2, t2, t0
        addi  t4, t4, 1
        j     while
fin:     mv    t2, t4
```

- **fin** encoded as displacement relative to current PC => **12**
 $PC = PC + 3 * 4$
- **while** encoded as displacement relative to current PC => **-16**
 $PC = PC + (-4) * 4$

Address	Content
0x0000100	li t0, 8
0x0000104	li t1, 4
0x0000108	li t2, 1
0x000010C	li t4, 0
0x0000110	bge t4, t1, 16
0x0000114	mul t2, t2, t0
0x0000118	addi t4, t4, 1
0x000011C	j -12
0x0000120	mv t2, t4

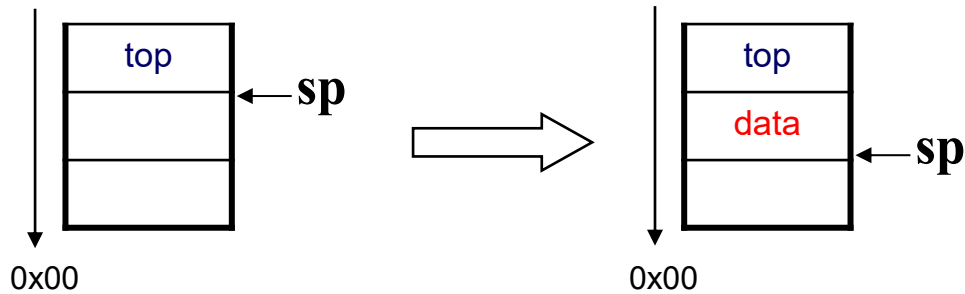
Stack addressing

- ▶ The Stack Pointer (SP):
 - ▶ Is a 32-bits register (4 bytes) in RISC-V₃₂
 - ▶ It stores the address of the top element on stack
 - ▶ Points to a word (4 bytes)
- ▶ Two types of operations:
 - ▶ push
 - ▶ pop

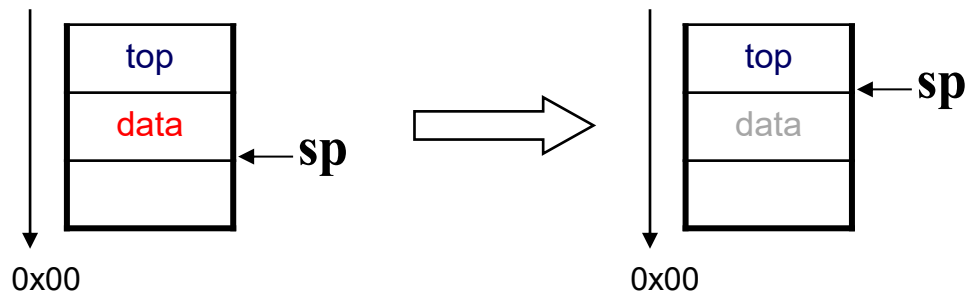


Stack grows towards lower addresses

PUSH Reg Stacks the contents of the record (data)



POP Reg Unstack registry contents (data)
Copies data to the Reg registry

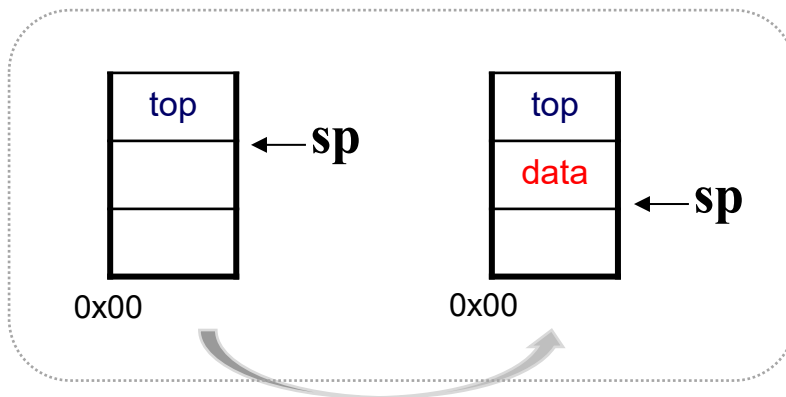


Stack addressing in RISC-V

- ▶ RISC-V does not have PUSH or POP instructions.
- ▶ The stack pointer register (`sp`) is visible to the programmer.
 - ▶ It will be assumed that the stack pointer points to the last element on the stack

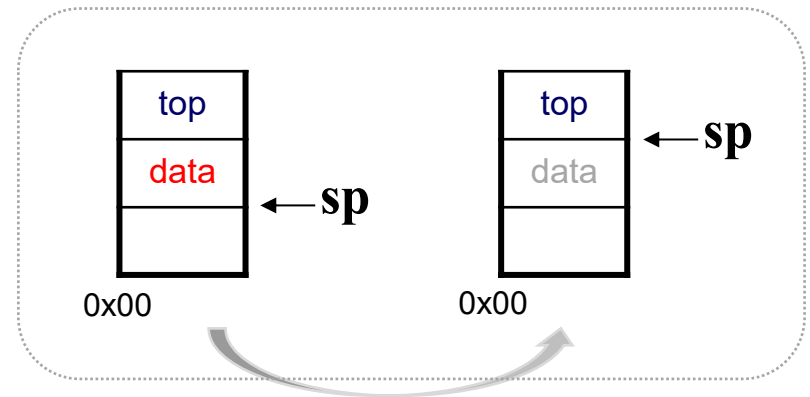
PUSH `t0`

```
addi sp, sp, -4  
sw    t0, 0(sp)
```



POP `t0`

```
lw    t0, 0(sp)  
addi sp, sp, 4
```



PUSH action in RISC-V₃₂

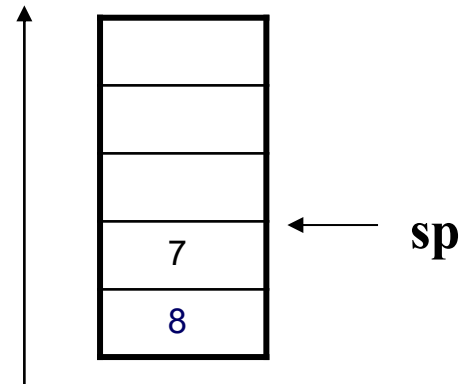
...

```
li    t2, 9
```

```
addi  sp, sp, -4
```

```
sw    t2, 0(sp)
```

...

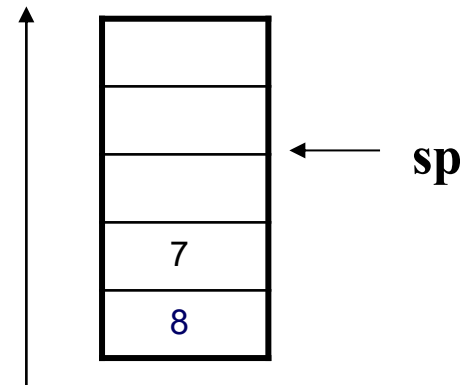


► Initial state:

- The stack pointer register (sp) points to the last element at the top of the stack.
- The t2 register stores the value 9

PUSH action in RISC-V₃₂

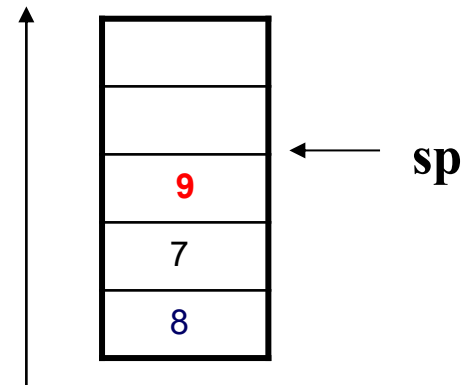
```
...  
li    t2, 9  
addi  sp, sp, -4  
sw    t2, 0(sp)  
...
```



- ▶ 4 is subtracted from the stack pointer register in order to insert a new word on the stack
 - ▶ `addi sp, sp, -4`

PUSH action in RISC-V₃₂

```
...  
li    t2, 9  
addi  sp, sp, -4  
sw    t2, 0(sp)  
...
```



- ▶ The contents of register t2 are inserted at the top of the stack:
 - ▶ `sw t2, 0(sp)`

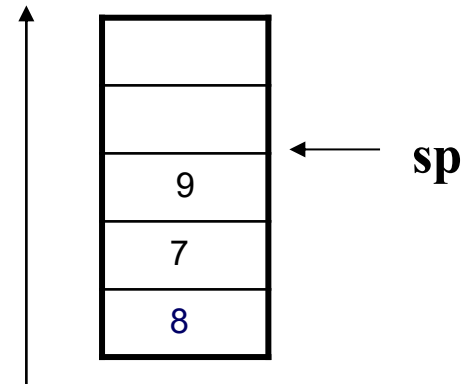
POP action in RISC-V₃₂

...

```
lw    t2, 0(sp)
```

```
addi  sp, sp, 4
```

...



- ▶ The data stored at the top of the stack is copied to t2 (9)
 - ▶ lw t2, 0(sp)

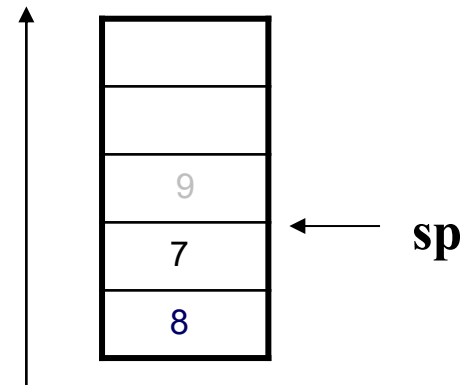
POP action in RISC-V₃₂

...

```
lw    t2, 0(sp)
```

```
addi sp, sp, 4
```

...

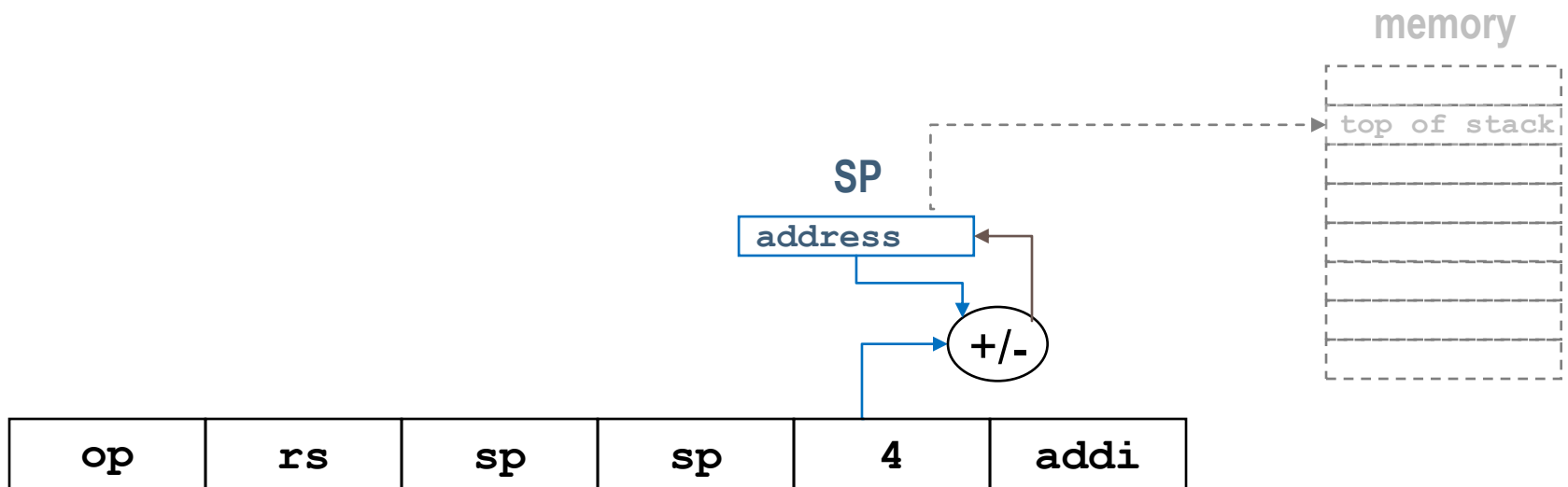


- ▶ The `sp` register is updated to point to the new top of the stack.
 - ▶ `addi sp, sp, 4`
- ▶ The unstacked data (9) is still in memory but will be overwritten in future PUSH (or similar memory access) operation.

Stack addressing in RISC-V

► Example: **push a0**

- `addi sp, sp, -4` # $SP = SP - 4$
- `sw a0, 0(sp)` # $\text{memory}[SP] = a0$



Exercise

- ▶ Indicate the type of addressing used in the following instructions RISC-V:
 1. `li t1, 4`
 2. `lw t0, 4(a0)`
 3. `bne x0, a0, label`

Exercise (solution)

1. **li t1, 4**

- ▶ **t1** -> direct to register
- ▶ **4** -> immediate

1. **lw t0, 4(a0)**

- ▶ **t0** -> direct to register
- ▶ **4(a0)** -> relative to base register

1. **bne x0, a0, label**

- ▶ **a0** -> direct to register
- ▶ **label** -> relative to program counter

Examples of addressing modes

- ▶ **la t0, label** **immediate**
 - ▶ The second operand of the instruction is an address
 - ▶ BUT this address is not accessed, the address itself is the operand
- ▶ **lw t0, label** **direct to memory (non- $\exists$ on RV32)**
 - ▶ The second operand of the instruction is an address
 - ▶ This address must be accessed in order to have the value to work with
- ▶ **bne t0, t1, label** **relative to PC register**
 - ▶ The third operand of the instruction is offset with respect to PC
 - ▶ label is encoded as a two's complement number representing the offset (as words) relative to the PC register

Examples of instructions

- ▶ `la t0, 0x0F000002`
 - ▶ Direct to register + immediate.
The `0x0F000002` value is loaded at `t0`
- ▶ `lbu t0, label(x0)`
 - ▶ Addresses direct to reg. + relative to base reg.
The byte at memory address `label` is loaded at `t0`
- ▶ `lb t0, 0(t1)`
 - ▶ Addresses direct to reg. + relative to base reg.
The byte in the memory position stored in `t1+0` is loaded in `t0`

Contents

1. Basic concepts on assembly programming
2. RISC-V 32 assembly, memory model and data representation
3. Floating point in assembly
4. Procedure calls and stack convention
5. Instruction formats, addressing modes, and instruction sets

Instruction sets

- ▶ Is **defined** by:
 - ▶ Instruction set
 - ▶ Instruction format
 - ▶ Registers
 - ▶ Addressing modes
 - ▶ Data types and formats

Instruction sets

- ▶ There are different ways for the **classification** of the instructions sets:
 - ▶ By complexity of the instruction set
 - ▶ CISC vs RISC
 - ▶ Execution modes
 - ▶ Stack
 - ▶ Register
 - ▶ Register-Memory, Memory-Register, ...

CISC vs RISC

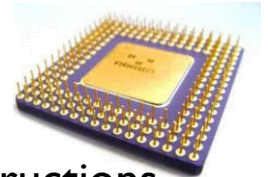
▶ *Complex Instruction Set Computer*

- ▶ Many instructions
- ▶ Complex instructions
 - ▶ More than one word
 - ▶ More complex control unit
 - ▶ Longer execution time
- ▶ Irregular design

▶ *Reduced Instruction Set Computer*

- ▶ Simple and orthogonal instructions:
 - ▶ Occupy one word
 - ▶ Instructions on registers
 - ▶ Use of the same addressing modes for all instructions (high degree of orthogonality)
- ▶ More compact design:
 - ▶ Easier and faster control unit
 - ▶ Space left over for more registers and cache memory

- ▶ About 20% of the instructions take up 80% of the total execution time of a program.
- ▶ 80% of the instructions are hardly ever used
- ▶ 80% of silicon underutilized, complex and costly



Execution modes

- ▶ The execution modes indicates the number of operands and the type of operands that can be specified in an instruction.
 - ▶ 0 addresses → Stack.
 - PUSH 5; PUSH 7; ADD
 - ▶ 1 address → Accumulator register.
 - ADD RI → $AC \leftarrow AC + RI$
 - ▶ 2 addresses → Registers, Register-memory, Memory-memory.
 - ADD .R0, .RI ($R0 \leftarrow R0 + RI$)
 - ▶ 3 addresses → Registers, Register-memory, memory-memory.
 - ADD .R0, .RI, .R2

ARCOS Group

uc3m | Universidad **Carlos III** de Madrid

L3: Fundamentals of assembler programming (5) Computer Structure

Bachelor in Computer Science and Engineering

Bachelor in Applied Mathematics and Computing

Dual Bachelor in Computer Science and Engineering and Business Administration

